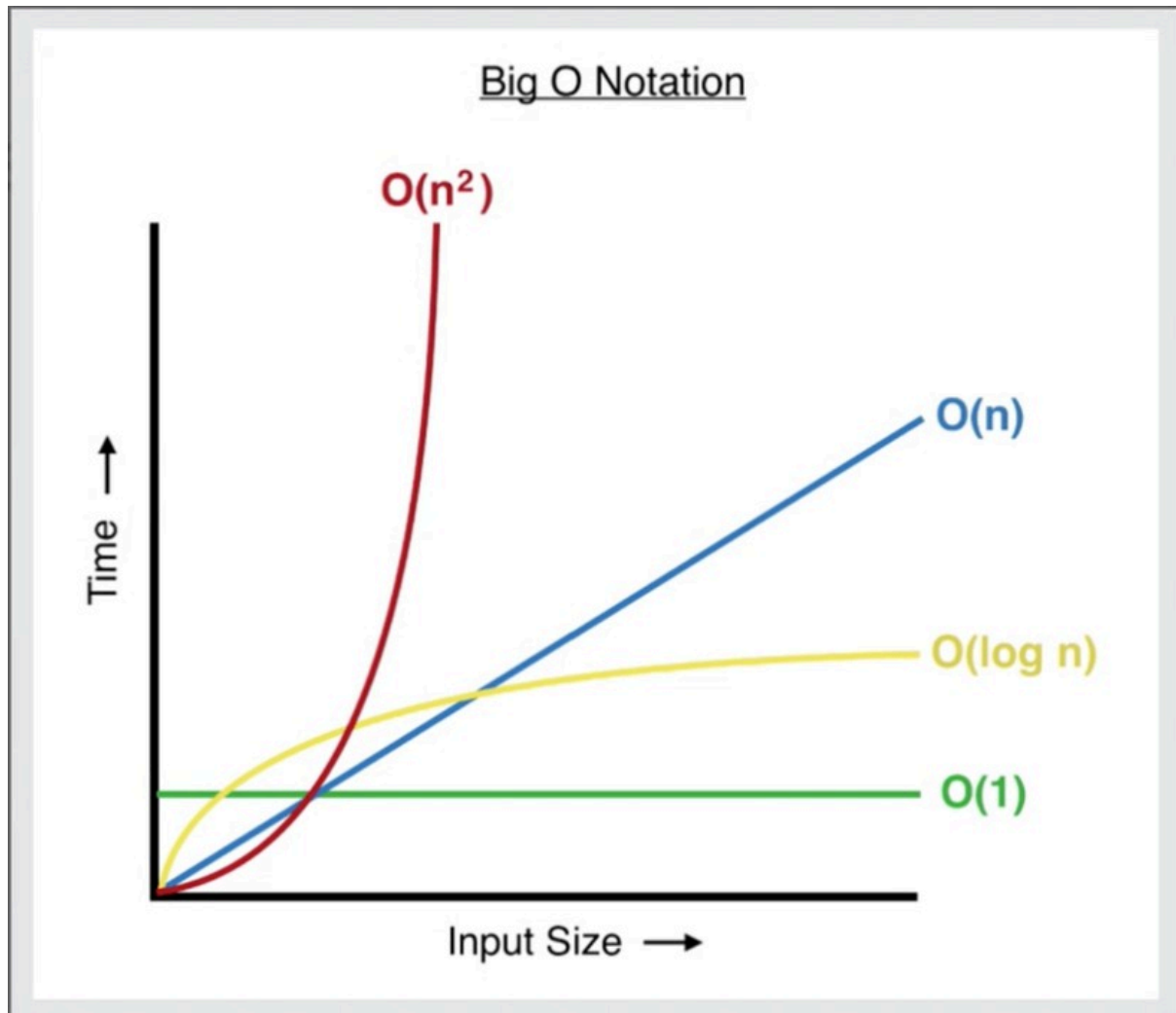


1. Big O

Big O (*THE*) It is a mathematical notation used to describe the upper limit of algorithmic complexity (running time or memory space) when the size of the input data (n) increases. It helps to compare the efficiency of algorithms in the worst-case scenario, ignoring small constants to focus on growth rate.



Some common complexity levels

$O(n)$: Linear complexity. Consider the following function to sum numbers from 1 to n :

```
int sum(int n)
{
    int s = 0;
    for(int i = 1; i <= n; i++)
        s += i;
    return s;
}
```

For each different value of n , the loop executes n times. Therefore, the execution time of the program depends on the input value of n . We can say that the complexity of the program is n . **$O(n)$** (actually $O(n+2)$).

$O(1)$: Constant complexity. It's also a problem of calculating the sum of numbers from 1 to n . Consider the following program segment:

```
int sum(int n)
{
    return n*(n+1)/2;
}
```

Unlike the summation function above, this program executes only one return statement for every input value. Therefore, we can say that the complexity of the above program is... **$O(1)$** .

$O(n^2)$: This is commonly encountered when there are two nested loops. Consider the following interchange sort function:

```
void interchangeSort(int *a, int n)
{
    for (int i = 0; i < n-1; i++)
        for (int j = i+1; j < n; j++)
            if (a[i] > a[j])
            {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
}
```

In this function, i will run from 0 to $n - 2$. For each value of i , the for loop will run $n - i - 1$ times (j from $i + 1$ to $n - 1$), so the number of comparisons to be performed will be $\Sigma(n - i - 1)$ (with i running from 0 to $n - 2$) = $(n - 1) + (n - 2) + (n - 3) + \dots + 2 + 1 = (n - 1)*n/2 = n^2/2 - n/2$. We say the complexity of the above algorithm is **$O(n^2)$** (actually it's $O(n^2/2 - n/2)$).

$O(\log n)$:

```
int binarySearch(int *arr, int n, int x) {
    int r = n - 1;
```

```

int l = 0;
while (r >= l)
{
    int mid = l + r) / 2;
    if (arr[mid] == x)
        return mid;
    if (arr[mid] > x)
        r = mid - 1;
    if (arr[mid] < x)
        l = mid + 1;
}

// If not found
return -1;
}

```

Simply put, this algorithm searches for an element with value x in an array `arr` sorted in ascending order. To find that value, the idea is to partition the array in half, examining the middle element. If that element is greater than the value being searched for, we examine the left half of that element; otherwise, we examine the right half, and so on until we find the element with the value being searched for (it might not be found if the value is not in the array). In the worst case, we would have to partition until the array contains only one element (if that is not the value being searched for, return -1). To reduce an array of n elements to a single element, we would need to perform $\log n$ (log base 2) partitions. Therefore, we can say that the above algorithm has a high complexity. **$O(\log n)$** .

Here are some points to consider when calculating algorithm complexity:

- When considering big O, we always consider n as an infinitely large number, so n can be considered as $+\infty$.(1)
- Execution steps that do not depend on input values (e.g., calculations, assignments, comparisons, etc.) have constant complexity. **$O(1)$** .(2)
- The sum of the constants is 1. **constant**.(3)
- The product of a positive number with $+\infty$ is also $+\infty$, The sum of a number with $+\infty$ is also $+\infty$.(4)

Steps to calculate algorithm complexity:

- Let T be the complexity of the algorithm we are looking for.
- Calculate the expression T by adding the execution times of the statements in the algorithm.
- Consider the term with the fastest growth rate as n approaches $+\infty$.
- Omit any omitted values according to the rules above.
- The value of the last remaining term is the required algorithmic complexity.

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
<u>Quicksort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$	$O(\log(n))$
<u>Mergesort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Timsort</u>	$\Omega(n)$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$
<u>Heapsort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n \log(n))$	$O(1)$
<u>Bubble Sort</u>	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Insertion Sort</u>	$\Omega(n)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Selection Sort</u>	$\Omega(n^2)$	$\theta(n^2)$	$O(n^2)$	$O(1)$
<u>Tree Sort</u>	$\Omega(n \log(n))$	$\theta(n \log(n))$	$O(n^2)$	$O(n)$
<u>Shell Sort</u>	$\Omega(n \log(n))$	$\theta(n(\log(n))^2)$	$O(n(\log(n))^2)$	$O(1)$
<u>Bucket Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$O(n^2)$	$O(n)$
<u>Radix Sort</u>	$\Omega(nk)$	$\theta(nk)$	$O(nk)$	$O(n+k)$
<u>Counting Sort</u>	$\Omega(n+k)$	$\theta(n+k)$	$O(n+k)$	$O(k)$
<u>Cubesort</u>	$\Omega(n)$	$\theta(n \log(n))$	$O(n \log(n))$	$O(n)$